

A Matrix-Free SBP-SAT Kernel: Improving Performance for GPU-enabled Earthquake Cycle Simulations

CS610

3-9-2026

Zac Cross

Presentation Outline:

1. Problem Introduction and Motivation
2. Background on Original SBP-SAT and CG Serial Design
3. Parallelization Strategy and Program Structure
4. Key Kernel Designs and Optimizations
5. Challenges and Solutions
6. Performance Evaluation

Earthquake Cycle Simulations

Goals:

- Simulate how the earth moves under different physical models and scenarios
- Feed simulation results into actionable policy in risk and hazard assessment

2 types of problems arise:

- Describing the physics correctly building accurate mathematical frameworks
- Building code that correctly implements the math and running the simulations
 - How to mesh? How to discretize the physics? etc
- We'll focus on #2

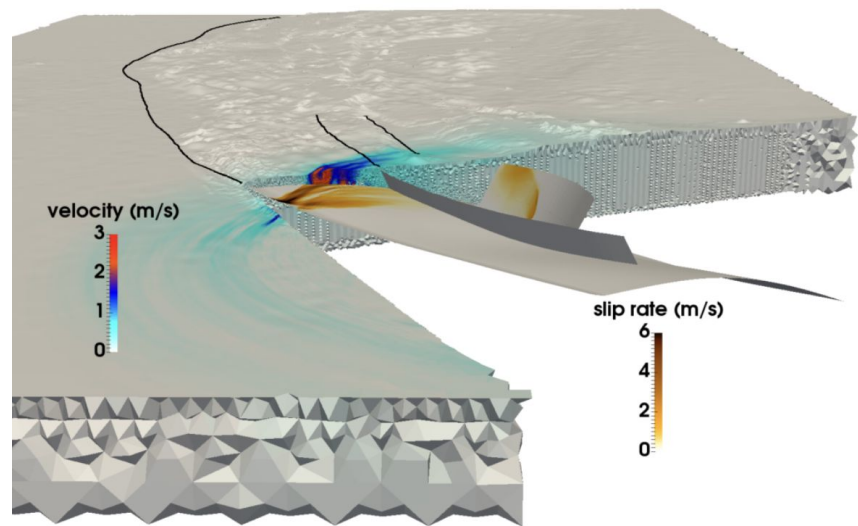
Motivating Simulations at Scale

Problems of interest involve:

- Scales with different orders of magnitude
 - Need to cover faults ~100s of km
 - Need material parameter resolution in microns
 - In higher dimensions, grids scale $O(n^d)$
- Governing Physics difficult to model efficiently
 - Many nonlinear friction laws
 - Time scales vary from millenia between events and Seconds for earthquakes.

Hazard Assessment needs accurate physics:

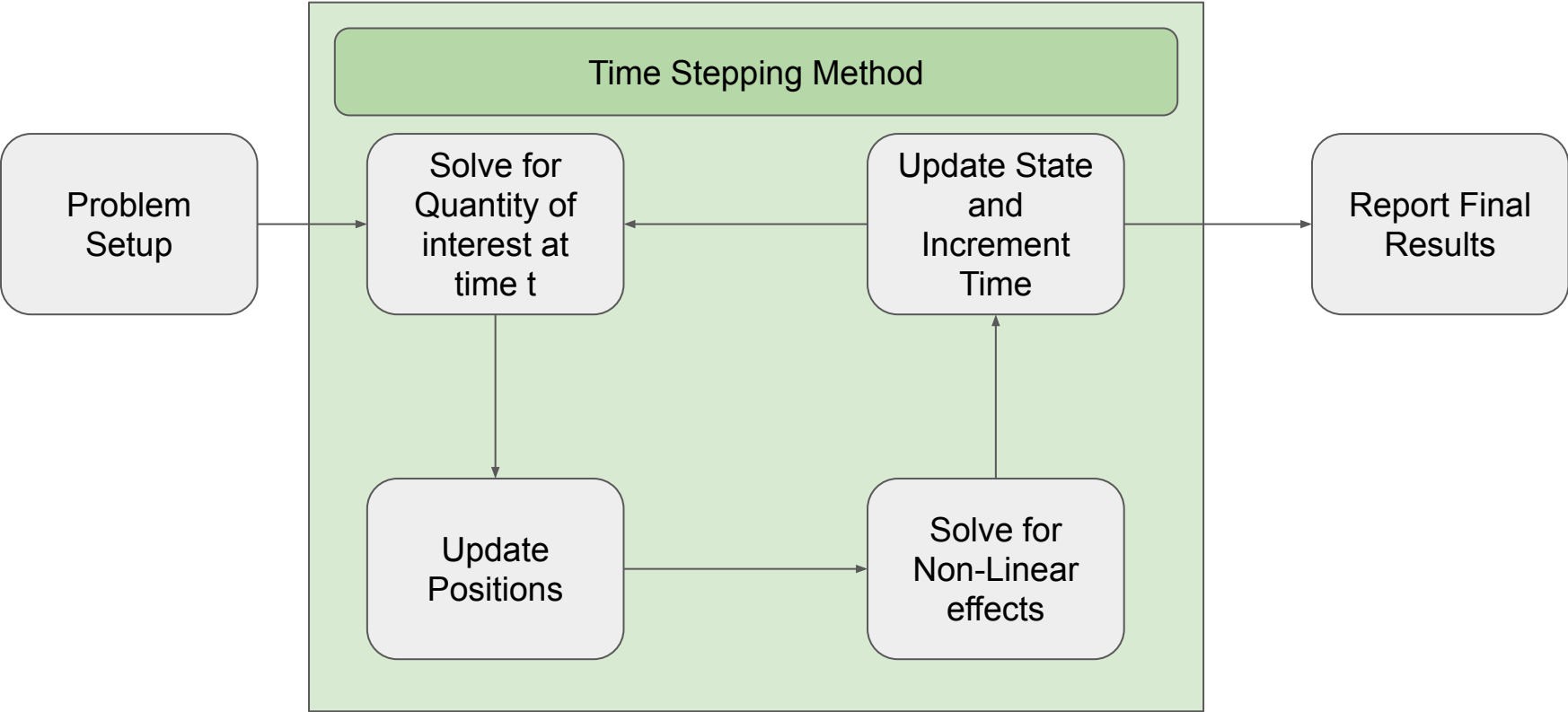
- Need to scale computation to do the above at meaningful scales
- GPUs are a big part of the effort to scale



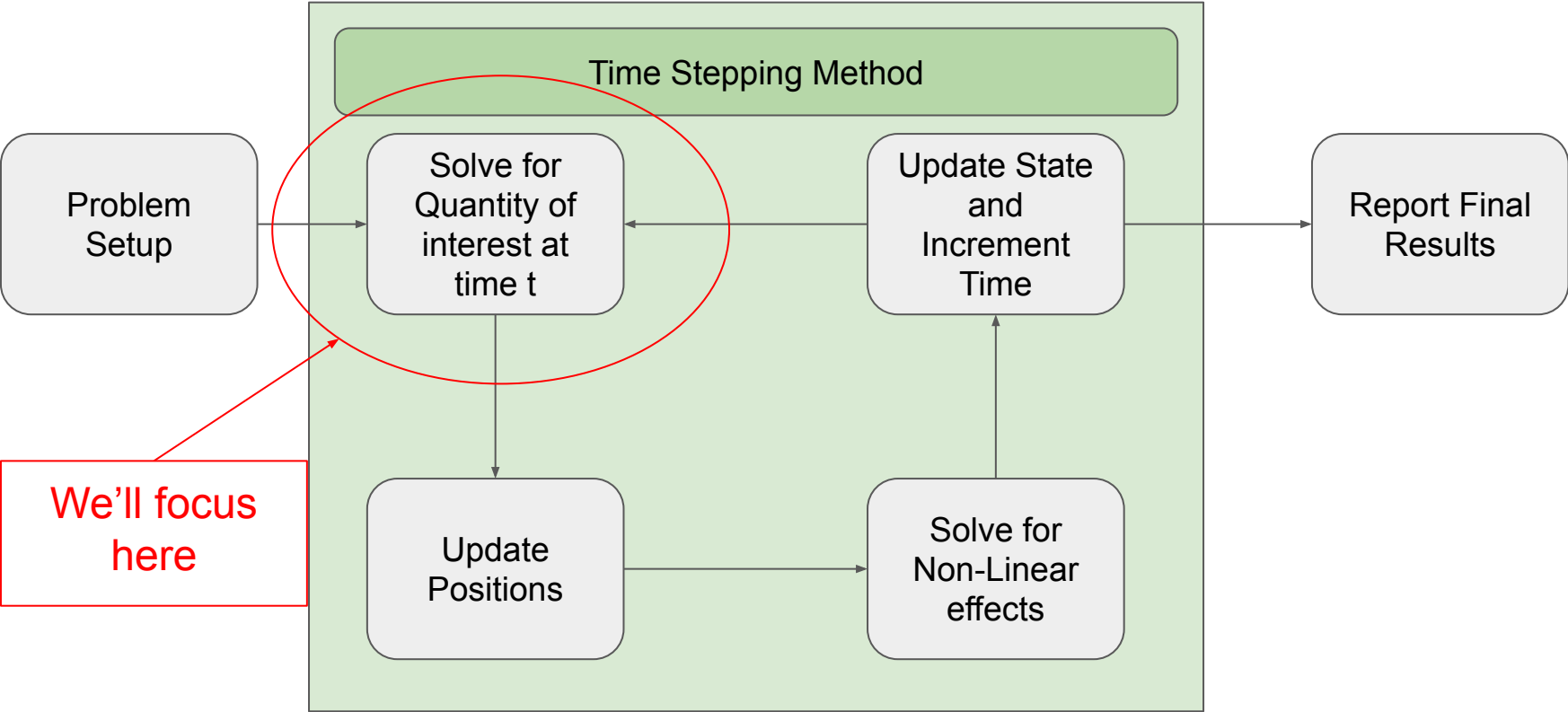
$$\rho \frac{\partial^2 u}{\partial t^2} = \frac{\partial}{\partial y} \left(\mu \frac{\partial u}{\partial y} \right), \quad y \in [-L, L], \quad t \geq 0,$$

Example of 1D Wave Equation for Linear Elasticity

Background: Typical Simulation Design Flow



Background: Typical Simulation Design Flow



Background: SBP-SAT Formulation and Theory

Summation by Parts with Simultaneous Approximation Term (SBP-SAT)

- High Order Finite Difference Method used to solve Ordinary and Partial Differential Equations
- SAT term weakly enforces boundary conditions
- Compatible with complex geometries and varying material parameters

Extra Math Background:

- Order of Accuracy:
 - How the approximation error decays as a function of grid spacing
 - $P = 2$ or 2nd order $\rightarrow$ Increase resolution by $2x$, then error goes down like factor of $2^2 = 4$
- Stability:
 - Discretization error on the operator is bounded, so the numerical method will converge

Background: SBP-SAT from a Computer Science Perspective

Operator Structure:

- The derivative operators are tridiagonal stencils
- High order operators largely the same
- Boundary SAT terms add non-repeating

Common Techniques to Make SBP-SAT Performant:

- Using Sparse Data structures
- Linear Solvers like Multigrid or conjugate gradient (CG)
- Preconditioning
- Incorporating GPUs for computations

$$\mathbf{D}_2 = \frac{1}{h^2} \begin{bmatrix} 1 & -2 & 1 & & & \\ 1 & -2 & 1 & & & \\ & 1 & -2 & 1 & & \\ & & \ddots & \ddots & \ddots & \\ & & & 1 & -2 & 1 \\ & & & & 1 & -2 & 1 \\ & & & & & 1 & -2 & 1 \end{bmatrix},$$

Example of 2nd derivative operator of order $p = 2$



Background: Conjugate Gradient for Large Linear Solves

Large Linear Solve:

- The goal is to set up a system with SBP-SAT to solve a Boundary Value Problem (BVP) for quantity of interest x , subject to boundary conditions b
- **$Ax = b$** , where A is the SBP operator, b is the boundary conditions vector, and x is a vector we want to solve for

Conjugate Gradient:

- Iterative method for linear solve
- Guaranteed convergence where A is Positive Definite
- Used for large systems

Serial Conjugate Gradient Design

Largely Serial Process:

- DAG of steps is fairly linear
- Fuse some operations to save memory and extra computation

Linear Algebra Operations:

- Vector Add
- Vector Scalar Multiply
- Vector Dot Product
- Matrix Vector Product

```

 $\mathbf{r}_0 := \mathbf{b} - \mathbf{A}\mathbf{x}_0$ 
if  $\mathbf{r}_0$  is sufficiently small, then return  $\mathbf{x}_0$  as the result
 $\mathbf{p}_0 := \mathbf{r}_0$ 
 $k := 0$ 
repeat
    
$$\alpha_k := \frac{\mathbf{r}_k^\top \mathbf{r}_k}{\mathbf{p}_k^\top \mathbf{A} \mathbf{p}_k}$$

     $\mathbf{x}_{k+1} := \mathbf{x}_k + \alpha_k \mathbf{p}_k$ 
     $\mathbf{r}_{k+1} := \mathbf{r}_k - \alpha_k \mathbf{A} \mathbf{p}_k$ 
    if  $\mathbf{r}_{k+1}$  is sufficiently small, then exit loop
    
$$\beta_k := \frac{\mathbf{r}_{k+1}^\top \mathbf{r}_{k+1}}{\mathbf{r}_k^\top \mathbf{r}_k}$$

     $\mathbf{p}_{k+1} := \mathbf{r}_{k+1} + \beta_k \mathbf{p}_k$ 
     $k := k + 1$ 
end repeat
return  $\mathbf{x}_{k+1}$  as the result
  
```

Parallelization Strategy

Conjugate Gradient is the Main Target:

- Algorithm is serial by nature
- Focus on Linear Algebra Kernels

Key Kernels for Project:

- Matrix Vector Product
- Vector Dot Product

```
 $\mathbf{r}_0 := \mathbf{b} - \mathbf{A}\mathbf{x}_0$   
if  $\mathbf{r}_0$  is sufficiently small, then return  $\mathbf{x}_0$  as the result  
 $\mathbf{p}_0 := \mathbf{r}_0$   
 $k := 0$   
repeat  
     $\alpha_k := \frac{\mathbf{r}_k^\top \mathbf{r}_k}{\mathbf{p}_k^\top \mathbf{A} \mathbf{p}_k}$   
     $\mathbf{x}_{k+1} := \mathbf{x}_k + \alpha_k \mathbf{p}_k$   
     $\mathbf{r}_{k+1} := \mathbf{r}_k - \alpha_k \mathbf{A} \mathbf{p}_k$   
    if  $\mathbf{r}_{k+1}$  is sufficiently small, then exit loop  
     $\beta_k := \frac{\mathbf{r}_{k+1}^\top \mathbf{r}_{k+1}}{\mathbf{r}_k^\top \mathbf{r}_k}$   
     $\mathbf{p}_{k+1} := \mathbf{r}_{k+1} + \beta_k \mathbf{p}_k$   
     $k := k + 1$   
end repeat  
return  $\mathbf{x}_{k+1}$  as the result
```

Parallelization Strategy

Conjugate Gradient is the Main Target:

- Algorithm is serial by nature
- Focus on Linear Algebra Kernels

Key Kernels for Project:

- Matrix Vector Product
- Vector Dot Product

$$\mathbf{r}_0 := \mathbf{b} - \mathbf{A}\mathbf{x}_0$$

if $\mathbf{r}_0$ is sufficiently small, then return $\mathbf{x}_0$ as the result

$$\mathbf{p}_0 := \mathbf{r}_0$$

$$k := 0$$

repeat

$$\alpha_k := \frac{\mathbf{r}_k^\top \mathbf{r}_k}{\mathbf{p}_k^\top \mathbf{A} \mathbf{p}_k}$$

$$\mathbf{x}_{k+1} := \mathbf{x}_k + \alpha_k \mathbf{p}_k$$

$$\mathbf{r}_{k+1} := \mathbf{r}_k - \alpha_k \mathbf{A} \mathbf{p}_k$$

if $\mathbf{r}_{k+1}$ is sufficiently small, then exit loop

$$\beta_k := \frac{\mathbf{r}_{k+1}^\top \mathbf{r}_{k+1}}{\mathbf{r}_k^\top \mathbf{r}_k}$$

$$\mathbf{p}_{k+1} := \mathbf{r}_{k+1} + \beta_k \mathbf{p}_k$$

$$k := k + 1$$

end repeat

return $\mathbf{x}_{k+1}$ as the result

Project Structure

Linear Algebra Operations

Matrix Free Adaptations

Conjugate Gradient Implementations

Final SBP-SAT Tests for Convergence

Key Kernels: Matrix Vector Product (Dense)

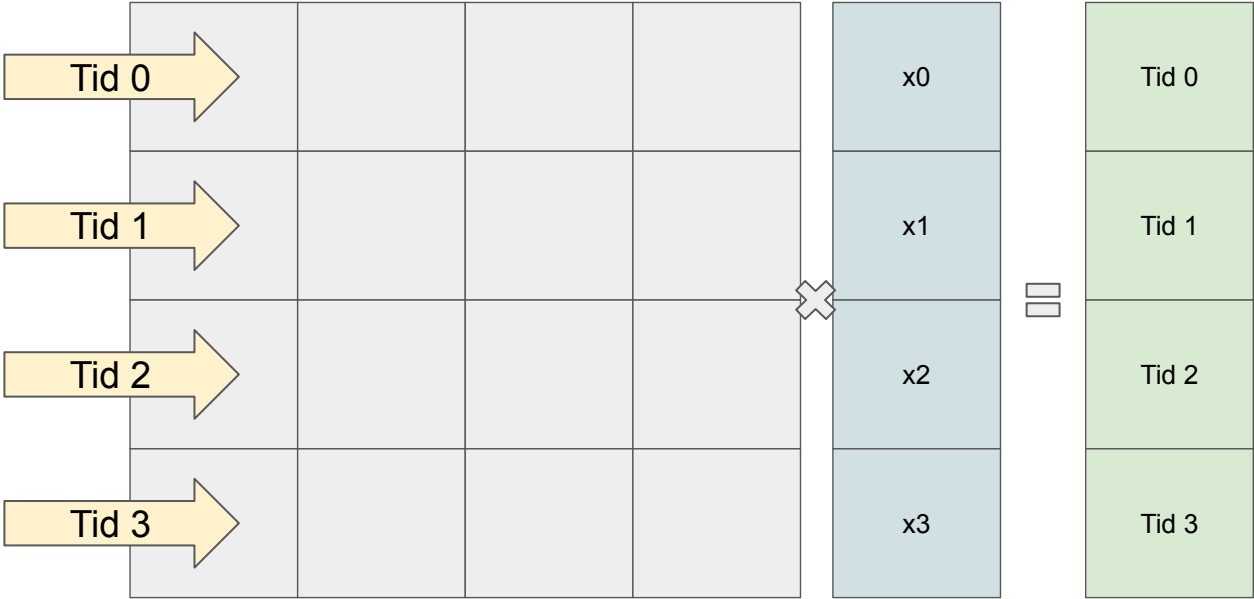
Solving $Ax = b$ for a vector b

Naive

- 1 Thread Takes does each dot product
- Launch Same # of threads as size of x

Memory Opt:

- Tile the x vector
- Let threads share the loads



Key Kernels: Matrix Vector Product (Dense)

```
__global__ void matrixVecCu(double *a, double *b, double *c, int m, int n) {  
  
    int idx_x = threadIdx.x + (blockDim.x * blockIdx.x);  
  
    if (idx_x < m){  
        double result = 0.0;  
        for (int i = 0; i < n; i++){  
            result += a[i + idx_x*n] * b[i];  
        }  
        c[idx_x] = result;  
    }  
}
```

```
__global__ void matrixVecCuOpt(double *a, double *b, double *c, int m, int n) {  
  
    // Assume Useful Shared Mem size is whole block ceil(blocksize/8) transactions per  
    extern __shared__ double block_mem[];  
  
    int glob_idx = threadIdx.x + (blockDim.x * blockIdx.x);  
    int local_idx = threadIdx.x;  
  
    int stride = blockDim.x;  
    double result = 0.0;  
  
    int tile_width = 0;  
    for (int offset = 0; offset < m; offset += stride){  
        // Load in shared part  
        if (offset + local_idx < m){  
            block_mem[local_idx] = b[local_idx + offset];  
        }  
        else {  
            block_mem[local_idx] = 0.0;  
        }  
        __syncthreads();  
  
        if (glob_idx < m){  
            tile_width = min(stride, m - offset);  
            for (int i = 0; i < tile_width; i++){  
                result += a[i + offset + glob_idx*n] * block_mem[i];  
            }  
            __syncthreads();  
        }  
        if (glob_idx < m){  
            c[glob_idx] = result;  
        }  
    }  
}
```

Key Kernels: Matrix Vector Product (Matrix Free)

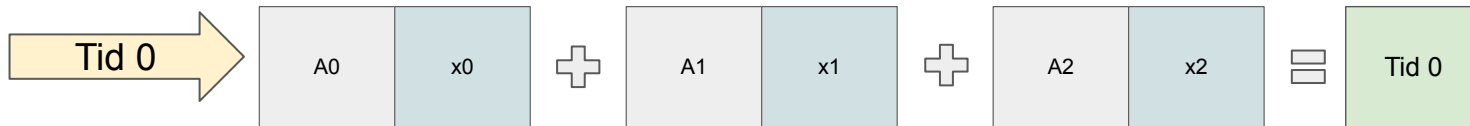
If A is a stencil, pull it out so each thread can ignore 0s

Hard Code the SBP operator and Boundary Term into the kernel

Less General, but more efficient

Register Opt:

- Reduce the number of operations and memory usage per thread



Key Kernels: Matrix Vector Product (Matrix Free)

```
_global_ void D2matrixFreeDirCu(double *x, double *result, int Nxp, double hi){  
  
    int bn, bn, bs;  
    int interior;  
    double d[3];  
    double bd[3];  
    double bsd[3];  
    bn = 1;  
    bn = 3;  
    bd[0] = 1.0; bd[1] = -2.0; bd[2] = 1.0;  
  
    bs = 3;  
    bsd[0] = 1.5; bsd[1] = -2.0; bsd[2] = 0.5;  
  
    interior = 3;  
    d[0] = 1.0; d[1] = -2.0; d[2] = 1.0;  
  
    int tid = threadIdx.x + (blockIdx.x * blockDim.x);  
  
    if (tid < Nxp){  
        result[tid] = 0.0; // Start by initializing result  
        // Lets start with D2  
        if (tid < bn){  
            for (int j = 0; j < bn; j++){  
                //printf("at index %u %u d2 = %.2f\n", i, j, bd[i*bn + j] / h2);  
                result[tid] += bd[j] + tid*bn * x[j];  
            }  
            if (tid == 0){  
                result[tid] = -0.5 * hi * result[tid]; // Makes it SPD  
            }  
            else {  
                result[tid] = -1.0 * hi * result[tid]; // Makes it SPD  
            }  
        }  
  
        else if (tid > Nxp - bn - 1){  
            for (int j = 0; j < bn; j++){  
                result[tid] += bd[(Nxp - tid - 1)*bn + (bn - j - 1)] * x[Nxp - j - 1];  
            }  
            if (tid == (Nxp-1)){  
                result[tid] = -0.5 * hi * result[tid]; // Makes it SPD  
            }  
            else {  
                result[tid] = -1.0 * hi * result[tid]; // Makes it SPD  
            }  
        }  
  
        else{ // Apply interior stencil  
            for (int j=0; j<interior; j++){  
                result[tid] += d[j]*x[tid+ j - (interior / 2)];  
            }  
            result[tid] = -1.0 * hi * result[tid]; // Makes it SPD  
        }  
    }  
}
```

```
_global_ void D2matrixFreeDirCuOpt(double *x, double *result, int Nxp, double hi){  
  
    double out = 0.0;  
  
    int tid = threadIdx.x + (blockIdx.x * blockDim.x);  
  
    if (tid < Nxp){  
        result[tid] = 0.0; // Start by initializing result  
        // Lets start with D2  
        if (tid==0){  
            out = (x[0] - 2.0 * x[1] + x[2]) * 0.5;  
            out += (1.5 + (2 * -13.0)) * x[0]; // SAT Term  
        }  
  
        else if (tid == Nxp-1){  
            out = (x[Nxp-3] - 2.0 * x[Nxp-2] + x[Nxp-1]) * 0.5;  
            out += (1.5 + (2 * -13.0)) * x[Nxp-1]; // SAT Term  
        }  
        else{ // Apply interior stencil  
            out = x[tid - 1] - 2.0 * x[tid] + x[tid + 1];  
        }  
  
        if (tid == 1){  
            out += -2.0 * x[0];  
        }  
        else if (tid == 2){  
            out += 0.5 * x[0];  
        }  
        else if (tid == Nxp-2){  
            out += -2.0 * x[Nxp-1];  
        }  
        else if (tid == Nxp-3){  
            out += 0.5 * x[Nxp-1];  
        }  
  
        out *= -1.0*hi;  
        result[tid] = out;  
    }  
}
```

Key Kernels: Matrix Vector Product (Matrix Free)

```
__global__ void D2matrixFreeDirCu(double *x, double *result, int Nxp, double hi){  
    int bm, bn, bs;  
    int interior;  
    double d[3];  
    double bd[3];  
    double bsd[3];  
    bm = 1;  
    bn = 3;  
    bd[0] = 1.0; bd[1] = -2.0; bd[2] = 1.0;  
  
    bs = 3;  
    bsd[0] = 1.5; bsd[1] = -2.0; bsd[2] = 0.5;  
  
    interior = 3;  
    d[0] = 1.0; d[1] = -2.0; d[2] = 1.0;  
  
    int tid = threadIdx.x + (blockIdx.x * blockDim.x);  
  
    if (tid < Nxp){  
        result[tid] = 0.0; // Start by initializing result  
        // Lets start with D2  
        if (tid < bm){  
            for (int j = 0; j < bn; j++){  
                //printf("at index %u %u d2 = %.2f\n", i, j, bd[i*bn + j] / h2);  
                result[tid] += bd[j + tid*bn] * x[j];  
            }  
        }  
    }  
}
```

```
__global__ void D2matrixFreeDirCuOpt(double *x, double *result, int Nxp, double hi){  
  
    double out = 0.0;  
  
    int tid = threadIdx.x + (blockIdx.x * blockDim.x);  
  
    if (tid < Nxp){  
        result[tid] = 0.0; // Start by initializing result  
        // Lets start with D2  
        if (tid==0){  
            out = (x[0] - 2.0 * x[1] + x[2]) * 0.5;  
            out += (1.5 + (2 * -13.0)) * x[0]; // SAT Term  
        }  
  
        else if (tid == Nxp-1){  
            out = (x[Nxp-3] - 2.0 * x[Nxp-2] + x[Nxp-1]) * 0.5;  
            out += (1.5 + (2 * -13.0)) * x[Nxp-1]; // SAT Term  
        }  
  
        else{ // Apply interior stencil  
            out = x[tid - 1] - 2.0 * x[tid] + x[tid + 1];  
        }  
    }  
}
```

Key Kernels: Vector Dot Product

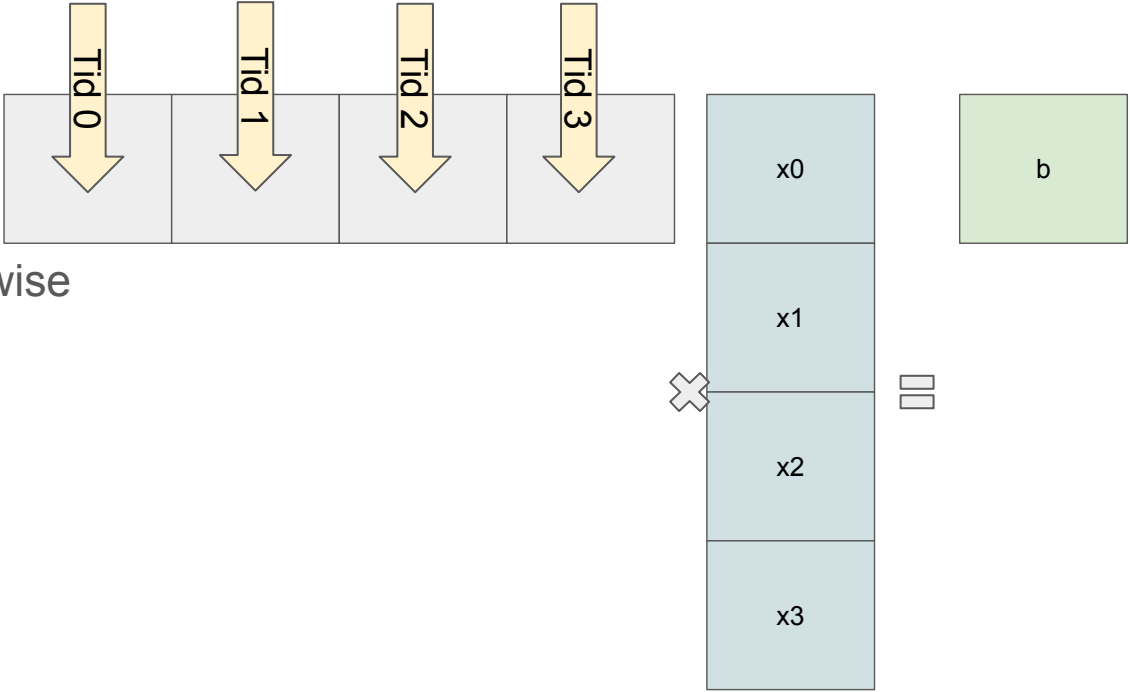
Solving $a^T x = b$ for a scalar b

Naive Approach:

- Each thread does an element wise product
- 1 Thread Sums and Write

Reduction Opt:

- Decreasing Stride Reduction +
Atomics



Key Kernels: Vector Dot Product

```
__global__ void VecDotCu(double *a, double *b, double *c, int m) {  
  
    int idx_x = threadIdx.x + (blockDim.x * blockIdx.x);  
    if (idx_x < m){  
        c[idx_x] = a[idx_x]*b[idx_x];  
    }  
}
```

```
__global__ void SumDotCu(double *c, int m) {  
  
    int idx_x = threadIdx.x + (blockDim.x * blockIdx.x);  
    double sum = 0.0;  
    if (idx_x < 1){  
        for (int i = 0; i < m; i++){  
            sum += c[i];  
        }  
        __syncthreads();  
  
        if (idx_x < 1){  
            c[0] = sum;  
        }  
    }  
}
```

```
__global__ void VecDotCuOpt(double *a, double *b, double *c, int m) {  
  
    extern __shared__ double block_mem[]; // should be size of block ...  
  
    int glob_idx = threadIdx.x + (blockDim.x * blockIdx.x);  
    int local_idx = threadIdx.x;  
  
    if (glob_idx < m){  
        block_mem[local_idx] = a[glob_idx]*b[glob_idx];  
    }  
    else {  
        block_mem[local_idx] = 0.0;  
    }  
  
    // Reduce each block  
    for (int stride = blockDim.x/2; stride >= 1; stride /= 2){  
        if (local_idx < stride){  
            block_mem[local_idx] += block_mem[local_idx + stride];  
        }  
        __syncthreads();  
    }  
  
    if (local_idx == 0){  
        atomicAdd(c, block_mem[0]);  
    }  
}
```

Key Challenges and Solutions

Ensuring Correctness between GPU and CPU Implementations

- Floating Point Arithmetic in different order can lead to close but different answers
- Solve with a matching tolerance proportional to problem size

Memory Bugs can be Tricky to Diagnose

- Compiling with CUDA support changes structure of heap allocation
- Careful attention to memory leaks is a must

Find the right Block Sizing:

- In this case, grid and block are 1D
- Found block size of 32 threads worked best for CG
- Finding the right dimensions to get best shared memory performance is key
- Tuned with a combination of trial and error and Nvidia Nsight Compute

Performance Evaluation: Methods

Linear Algebra Operations:

- Random Dense Matrix and Vector
- Timed CPU vs Naive vs Optimized
- Validated Against CPU for Correctness
- Running on Array Sizes 1000 - 32000
- **Reporting the Time to execute the kernel, excluding Data Transfer**
- $\text{Speed Up} = \text{CPU Time} / \text{GPU Time}$

Conjugate Gradient Methods

- Timed for SBP-SAT Solve
- Validated Against CPU for correctness
- Measured Error for Operator Convergence
- **Including Data Transfer in Timing**
- Limit to 10 iterations for testing speed
- Arrays from 100 - 3200 elements for dense tests
- 100000 - 3200000 elements for matrix free

Note of Convergence Testing

1D Scalar Wave Equation:

- Simplified version of Linear Elasticity from Earthquake Modeling

$$u_{tt}(x) = u_{xx} + \text{source term}$$

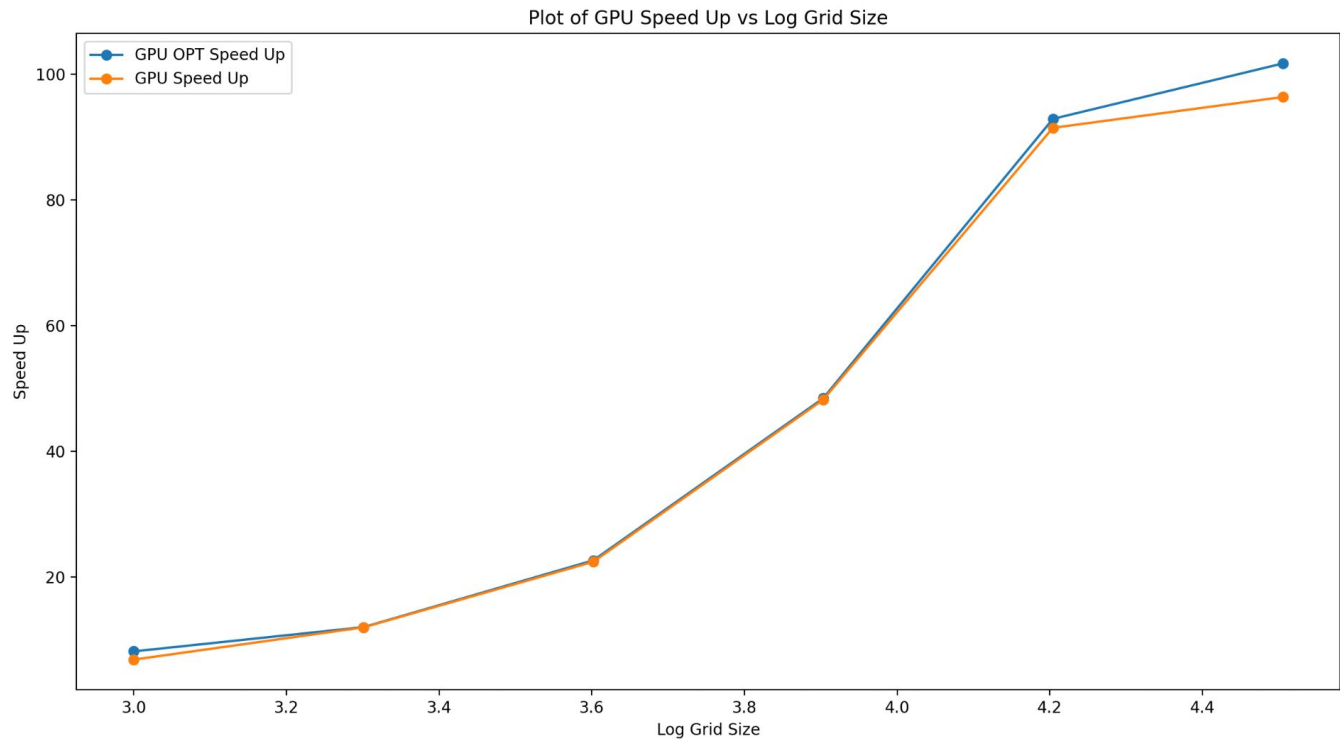
- Double Subscript = 2nd derivative
- On domain $[-1, 1]$ with grid spacings from 100 - 3200
- For simplicity let $u_{tt}(x) = 0$.

$$0 = u_{xx} + \text{source term}$$

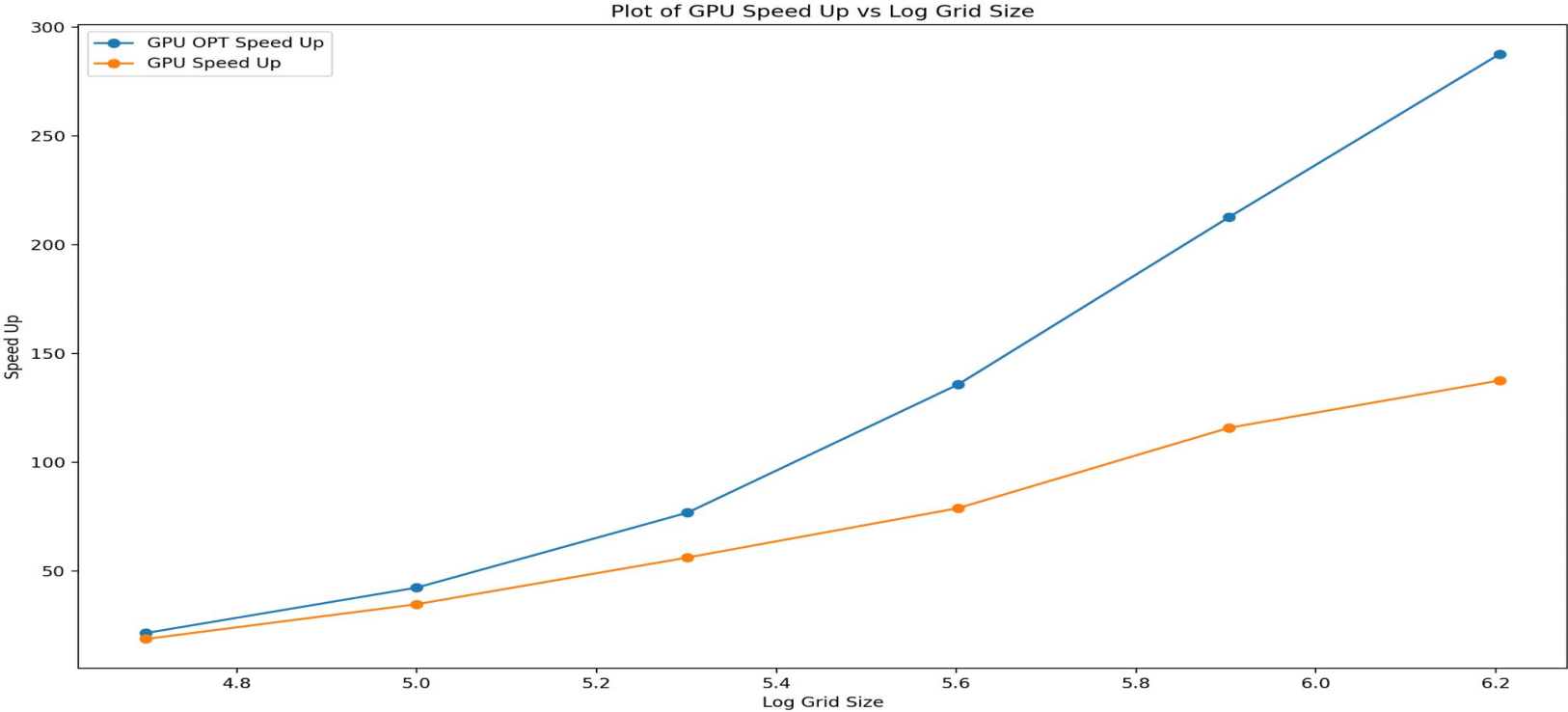
Method of Manufactured Solution

- For testing, choose solution to be $f(x) = \sin(\pi x) + 1$
- Measure error of CG solution from exact solution with L2 error.

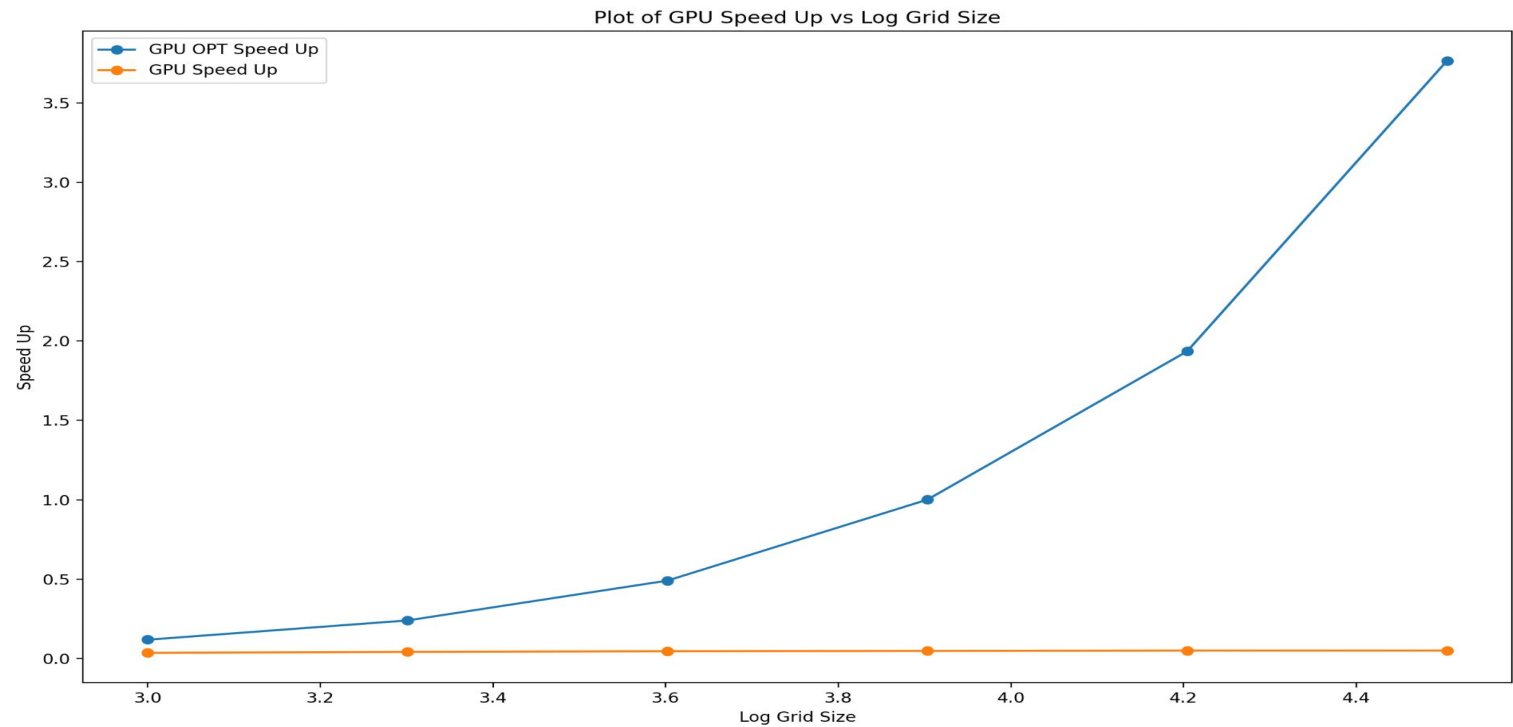
Results: Matrix Vector Product Dense



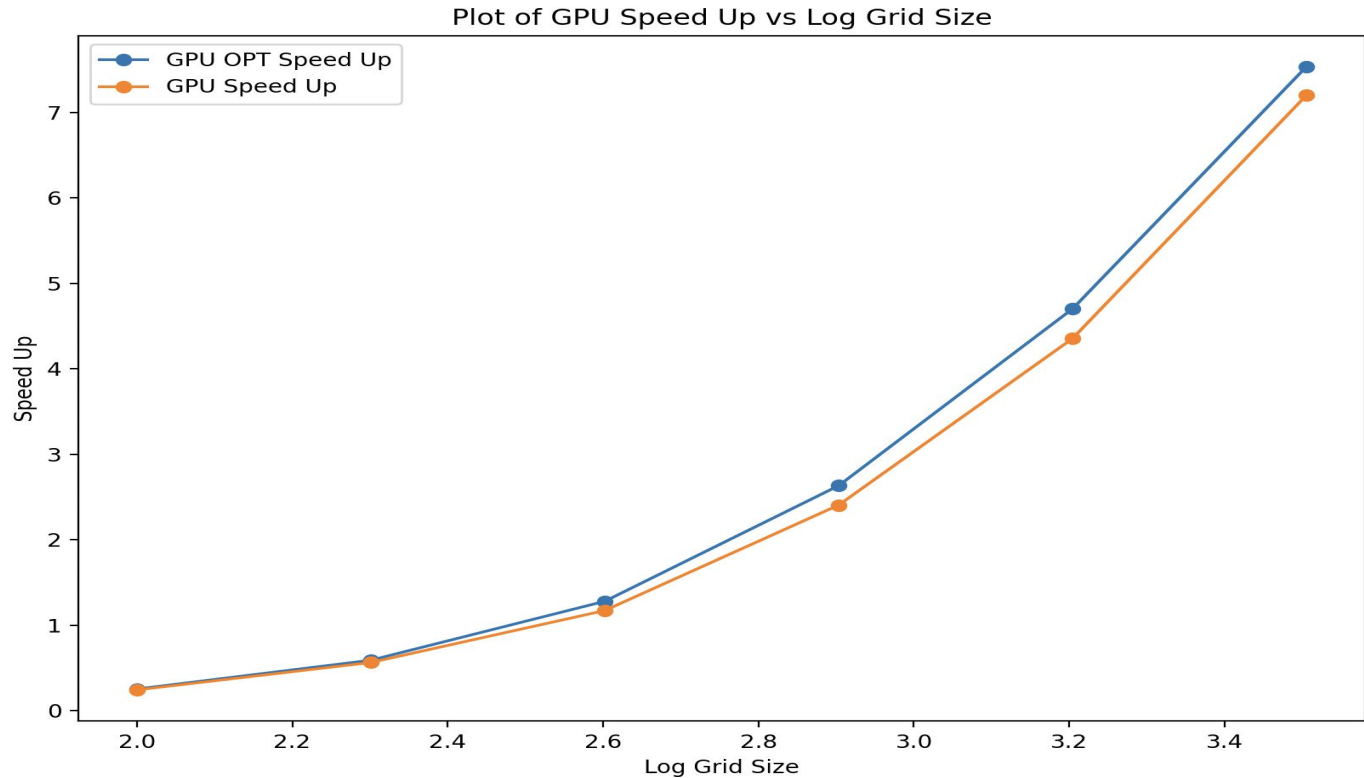
Results: Matrix Vector Product Matrix Free



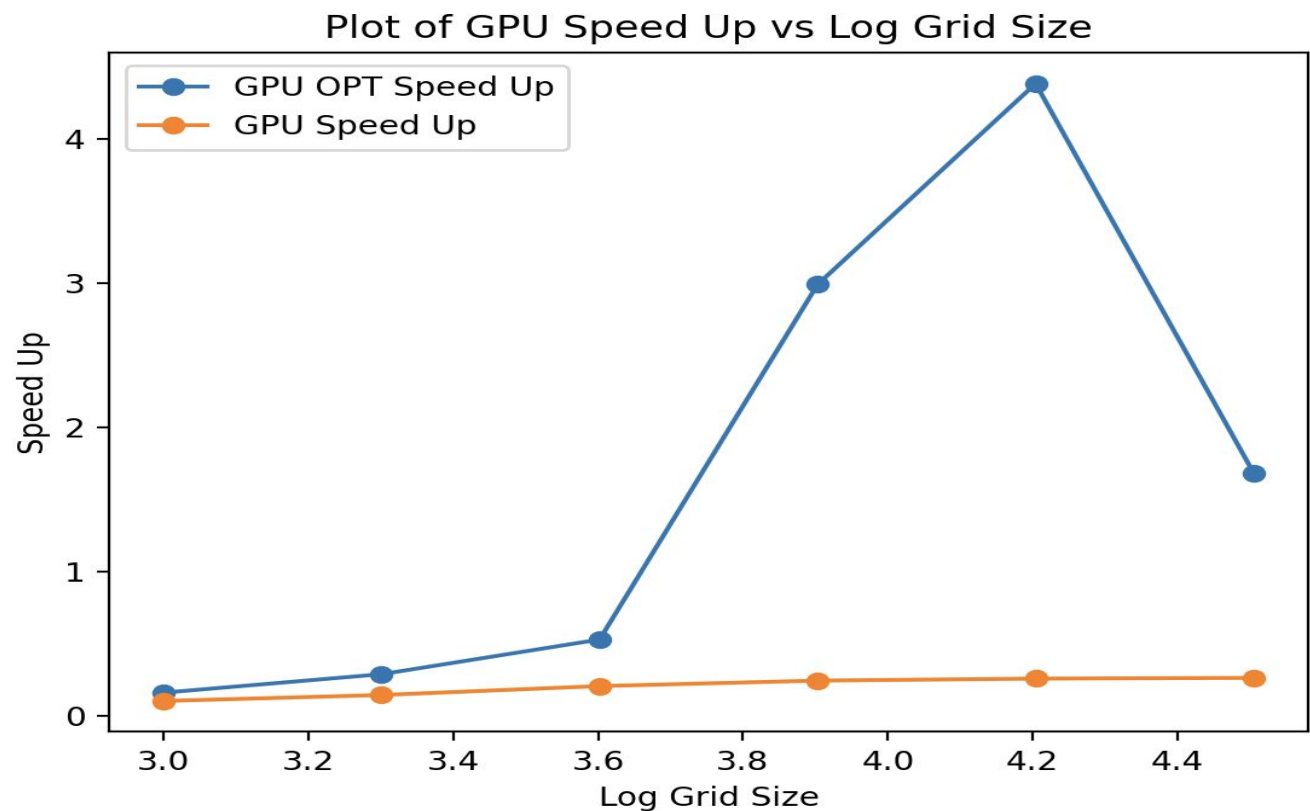
Results: Vector Dot Product



Conjugate Gradient Dense



Conjugate Gradient Matrix Free



* CPU comparison is also MF

**Both implementations >10x faster than Dense GPU code

Thanks!